

Structured Streaming – Grundkonzepte

Structured Streaming ist die Streaming-Engine von Apache Spark. Sie ermöglicht es, kontinuierlich eintreffende Daten (z. B. Log-Dateien, Sensordaten, Nachrichten aus einer Message Queue) nahezu in Echtzeit zu verarbeiten – und zwar mit denselben APIs, die man auch für klassische Batch-Verarbeitung nutzt. Das macht Streaming-Anwendungen deutlich einfacher zu entwickeln, da man nicht zwei komplett unterschiedliche Programmiermodelle lernen muss.

Was bedeutet "nahezu in Echtzeit mit Exactly-Once-Garantie"?

Structured Streaming ist eine "near real-time"-Verarbeitungs-Engine mit End-to-End-Fehlertoleranz und sogenannten "Exactly-once"-Garantien. Das bedeutet: Selbst wenn während der Verarbeitung ein Fehler auftritt (z. B. ein Rechenknoten fällt aus), sorgt das System dafür, dass jeder Datensatz am Ende genau einmal korrekt verarbeitet wird – nicht doppelt und nicht gar nicht.

Datenquellen (Sources)

Structured Streaming kann Daten aus verschiedenen Quellen kontinuierlich einlesen:

- **Auto Loader:** Liest automatisch neue Dateien aus einem Cloud-Speicher ein, sobald sie dort ankommen.
- **Delta-Lake-Tabellen:** Delta-Tabellen unterstützen selbst das Streaming, sodass man Änderungen an einer Tabelle direkt als Stream weiterverarbeiten kann.
- **Standard-Konnektoren:** Für gängige Message-Systeme und Warteschlangen (z. B. Kafka) gibt es fertige Anbindungen.
- **Steuerung der Micro-Batch-Größe:** Man kann konfigurieren, wie viele Daten pro Verarbeitungsschritt maximal gelesen werden.

Wie Daten verarbeitet und ausgeliefert werden

Checkpoints

Ein Checkpoint speichert den aktuellen Verarbeitungsstatus einer Streaming-Abfrage dauerhaft ab. Fällt die Anwendung aus und wird neu gestartet, kann sie anhand des Checkpoints genau dort

weitermachen, wo sie aufgehört hat – das ist die Grundlage für die Fehlertoleranz von Structured Streaming.

Output Modes (Ausgabemodi)

Es gibt drei Modi, die bestimmen, wie Ergebnisse an die Zielsenke geschrieben werden:

Modus	Bedeutung
Append	Es werden nur neu hinzugekommene Zeilen ausgegeben
Update	Nur die Zeilen, die sich seit dem letzten Schritt geändert haben, werden ausgegeben
Complete	Das komplette, aktuelle Ergebnis wird bei jedem Schritt neu ausgegeben

Trigger-Intervalle

Über Trigger-Intervalle steuert man, wie oft neue Daten verarbeitet werden. Das beeinflusst direkt die Latenz (wie schnell neue Daten sichtbar werden) und die Kosten (wie oft Rechenressourcen aktiv sind). Für besonders zeitkritische Anwendungen gibt es außerdem einen Real-Time-Modus mit einer Latenz von bis zu 5 Millisekunden.

Zustandsverarbeitung (State Management)

Je nach Art der Abfrage unterscheidet man:

- **Zustandslose (stateless) Abfragen:** Jeder neue Datensatz wird unabhängig von vorherigen verarbeitet, es muss kein Zwischenzustand gespeichert werden – z. B. ein einfacher Filter.
- **Zustandsbehaftete (stateful) Operationen:** Hierbei muss sich das System etwas "merken", zum Beispiel bei Aggregationen (Summen über Zeit), Verknüpfungen (Joins) zwischen zwei Streams oder beim Entfernen von Duplikaten (Deduplication).

Watermarks

Ein Watermark legt fest, wie lange Structured Streaming auf verspätet eintreffende Daten wartet, bevor ein Zeitfenster als "abgeschlossen" gilt und die zugehörigen Zwischenergebnisse aus dem Speicher entfernt werden. Das ist wichtig, um den Speicherverbrauch bei zustandsbehafteten Operationen (z. B. Aggregationen über Zeitfenster) in einem vernünftigen Rahmen zu halten.

Überwachung (Monitoring)

- **StreamingQueryListener:** Eine Schnittstelle, über die man den Fortschritt und die Performance einer laufenden Streaming-Abfrage programmatisch verfolgen kann.
- **Unity Catalog:** Wird auch für Streaming-Datenquellen und -Ziele zur Governance und Zugriffskontrolle eingesetzt, sodass dieselben Sicherheitsregeln wie bei Batch-Daten gelten.

Warum Structured Streaming wichtig ist

Für viele reale Anwendungsfälle – etwa das Überwachen von Sensordaten, die Verarbeitung von Log-Dateien oder das Nachladen neuer Dateien aus einem Data Lake – reicht klassische Batch-Verarbeitung nicht aus, weil Daten kontinuierlich eintreffen. Structured Streaming schließt diese Lücke, ohne dass man dafür ein komplett neues Programmiermodell lernen muss: Die gleichen DataFrame-Operationen wie im Batch-Betrieb funktionieren größtenteils auch im Streaming-Kontext.

Quelle: <https://docs.databricks.com/aws/en/structured-streaming/concepts>

Stand: 2026-08-06